

TRAINING SESSION PLAN

Wire Framework Developer Onboarding.

A hands-on, half-day workshop for analytics engineers, data engineers and consultants new to Wire. Participants follow along in their own environment, install the plugin, scaffold an engagement, and complete a full generate → validate → review lifecycle end-to-end.

FORMAT Workshop Hands-on, follow-along	DURATION 3.5 hrs Half-day, single session	AUDIENCE Engineers Analytics, data, consulting	PRE-REQS Claude Code + Wire plugin access
--	---	--	---

8 PARTS From "what is Wire" through autopilot & next steps.	4 HANDS-ON EXERCISES Install · scaffold · lifecycle · release-type selection.	7 LEARNING OUTCOMES Measurable; see §Assessment on page 4.
---	---	--

DOCUMENT CONTENTS

§1	Learning objectives	p. 2
§2	Prerequisites	p. 2
§3	Session agenda · Parts 1–4	p. 3
§4	Session agenda · Parts 5–8	p. 4
§5	Assessment & post-session support	p. 5
§6	Appendix A · Troubleshooting	p. 5

AT A GLANCE

- Install plugin → restart Claude Code → verify with `/wire:help`.
- Scaffold engagement with `/wire:new`; pick one of 7 release types.
- Run generate → validate → review for at least two artifact types.
- Read `status.md` and `execution_log.md` to interpret state.

BRING WITH YOU

A test SOW or sample PDF, plus a GitHub repo or local directory for the engagement.

§1 Learning objectives

By the end of this session, participants will be able to...

NO.	OUTCOME	COVERED IN
L0-1	Install and configure the Wire Framework plugin inside Claude Code.	Part 2 · Installation
L0-2	Create a new engagement and delivery release with <code>/wire:new</code> .	Part 2 · Installation
L0-3	Run the generate → validate → review lifecycle for at least two artifact types.	Part 3 · Lifecycle
L0-4	Explain the difference between release types and choose the right one for a given SOW.	Part 4 · Release types
L0-5	Understand automatic state management — how the engagement-context skill, execution log and <code>status.md</code> work together.	Part 5 · Context & state
L0-6	Use <code>/wire:plan</code> for structured session planning, and navigate the execution log.	Part 5 · Context & state
L0-7	Know when and how to use Autopilot for end-to-end autonomous delivery.	Part 7 · Autopilot

§2 Prerequisites

REQUIRED BEFORE THE SESSION

- ☐ Claude Code CLI installed — `npm install -g @anthropic-ai/claude-code` or the desktop app.
- ☐ Access to the `rittmmananalytics/wire` plugin on GitHub (repo access granted).
- ☐ A test client SOW or a sample PDF to use as source material.
- ☐ A GitHub repo (or local directory) to use as the engagement repo.

RECOMMENDED

- Familiarity with dbt projects — needed for the Development section.
- Basic Git knowledge — branches, commits, remotes.

ENVIRONMENT CHECK

Run the following at least 24 hours before the session and report any failures to `#wire-framework`.

Claude Code	<code>claude --version</code>
Plugin marketplace	<code>/plugin marketplace list</code>
Wire installed	<code>/wire:help</code>
Repo writeable	<code>git status</code> in target dir

MATERIALS

Slides and reference docs ship with the plugin. See `USER_GUIDE.md` in the repo, or `wire/dist/wiki/User-Guide.md` on the GitHub wiki.

§3 Session agenda · Parts 1–4

Each part has an objectives bullet, the activities run in order, and reference materials are listed in the right rail. Times are wall-clock and include short breaks between parts.

01 PART ONE	What is Wire and why does it exist? Objectives · Understand the problem Wire solves; the core architecture (specs, commands, skills, releases); see the framework in action with a live demo. – Presentation — the two problems Wire solves: consistency + context. 10 min – Live demo — run <code>/wire:help</code> and walk through the command tree. 5 min – Discussion — how does this differ from prompting an AI directly? 5 min	20 min SLIDES PART 1 DECK USER_GUIDE §1–3
02 PART TWO	Installation and first engagement. Objectives · Install the plugin and verify it works; create a first engagement with <code>/wire:new</code> ; understand the <code>.wire/</code> directory structure. – Hands-on — install the Wire plugin. 10 min · Exercise 1 – Hands-on — run <code>/wire:new</code> on a test repo. 15 min · Exercise 2 – Walkthrough — explore the generated <code>.wire/</code> structure. 5 min	30 min EXERCISES EX 1 · PLUGIN INSTALL EX 2 · FIRST ENGAGEMENT
03 PART THREE	The artifact lifecycle — generate, validate, review. Objectives · Understand generate → validate → review as a repeatable gate pattern; run a full requirements cycle end-to-end; read and interpret <code>status.md</code> and <code>execution_log.md</code> . – Presentation — how the three gates work and why. 10 min – Hands-on — run requirements generate / validate / review. 25 min · Exercise 3 – Walkthrough — read the resulting <code>status.md</code> and <code>execution_log.md</code>. 10 min	45 min EXERCISES EX 3 · FULL LIFECYCLE USER_GUIDE §6
04 PART FOUR	Release types — choosing the right one. Objectives · Know all seven release types and their in-scope artifact sets; map a given SOW to the correct release type; understand discovery releases and the Shape Up pitch → brief → spawn flow. – Presentation — release type comparison table. 10 min – Group exercise — given three SOW descriptions, pick a release type. 10 min · Exercise 4 – Q&A — edge cases (dashboard-first vs full platform, when to use discovery). 5 min	25 min EXERCISES EX 4 · SELECTION SCENARIOS USER_GUIDE §4

§4 Session agenda · Parts 5–8

05

PART FIVE

Automatic context and state management.

20 min

Objectives · Understand how the engagement-context skill replaces explicit session commands; what the execution log captures and how to read it; when to use

`/wire:plan`.

- Presentation — how state flows automatically (engagement-context skill → `status.md` → `execution_log.md`). 10 min
- Demo — open a repo mid-engagement and see context load automatically. 5 min
- Discussion — when would you use `/wire:plan`? 5 min

REFERENCE
`USER_GUIDE` §3
SESSION
LIFECYCLE

06

PART SIX

Skills — ad-hoc development outside Wire commands.

20 min

Objectives · Know which skills exist and when they auto-activate; understand how skills complement Wire commands for day-to-day development work.

- Presentation — skills architecture: triggers, activation, self-logging. 10 min
- Demo — open a dbt model file and observe the dbt-development skill activating. 5 min
- Demo — open a LookML file and observe lookml-content-authoring activating. 5 min

REFERENCE
`USER_GUIDE` §6
`SKILLS/README.MD`

07

PART SEVEN

Autopilot and advanced features.

25 min

Objectives · Know when Autopilot is appropriate and when to run commands manually; understand MCP integrations (Fathom, Jira/Linear, Confluence/Notion); know how to extend the framework.

- Presentation — Autopilot: what it does, when to trust it, how to recover if blocked. 10 min
- Presentation — MCP integrations overview. 5 min
- Discussion — customisation: adding commands, modifying specs, client-level conventions. 10 min

REFERENCE
`USER_GUIDE` §17
`USER_GUIDE` §22

08

PART EIGHT

Q&A and next steps.

15 min

Objectives · Clear any remaining questions; agree on first real-engagement use.

- Open Q&A. 10 min
- Each participant commits to one engagement they'll use Wire on next. 5 min

MATERIALS
QUICK
REFERENCE
CARD
SUPPORT
CHANNELS

TOTAL WALL-CLOCK

Parts 1–8 sum to **3h 20m** of facilitator time. Add a 10-minute mid-session break between Parts 4 and 5 to hit the published 3.5-hour duration.

§5 Assessment & post-session support

HOW TO MEASURE LEARNING OUTCOMES

Tick the criteria below for each participant during or immediately after the session. Anyone unticked on three or more gets a 1:1 follow-up before their next engagement.

- ☐ Participant can install the plugin and run `/wire:new` independently.
- ☐ Participant can explain the three-gate lifecycle without prompting.
- ☐ Participant can select the correct release type for a given SOW description.
- ☐ Participant can read `execution_log.md` and describe what happened in a previous session.
- ☐ Participant knows which skill activates for dbt work and why.

POST-SESSION RESOURCES

User Guide `wire/dist/wiki/User-Guide.md` · or `USER_GUIDE.md` in the repo

Quick reference `wire/docs/enablement/wire_developer_quick_reference.md`

Release notes `wire/docs/CHANGELOG.md`

FOLLOW-UP CADENCE

- **Office hours** — one week post-training. Bring your first real engagement for review.
- **Slack** — `#wire-framework` for day-to-day questions.
- **Issues** — raise bugs or spec improvements at `github.com/rittmananalytics/wire/issues`.

§6 Appendix A · Troubleshooting common issues

ISSUE	LIKELY CAUSE	FIX
Plugin commands not appearing	Plugin not installed, or Claude Code not restarted after install.	Re-run install steps; fully restart Claude Code; re-check with <code>/wire:help</code> .
<code>status.md</code> not updating	Wrong release folder path passed to the command.	Check the folder name matches a subdirectory of <code>.wire/releases/</code> exactly.
Engagement-context skill not firing	No <code>.wire/</code> directory in the current repo, or you're outside the engagement root.	Run <code>/wire:new</code> first, or <code>cd</code> into the engagement root and retry.
Validation failing unexpectedly	Source materials missing from <code>requirements/</code> .	Add the SOW and call transcripts to the engagement folder, then re-run generate before validate.
Autopilot blocked mid-run	An artifact requiring human input has been hit.	Fix the blocking artifact manually, mark it <code>approved</code> in <code>status.md</code> , then re-run Autopilot.

STILL STUCK?

Open a ticket in `github.com/rittmananalytics/wire/issues` with the failing command, the relevant slice of `execution_log.md`, and your release ID. The framework team triages within one working day.